University of Victoria

ECE 455 – Real Time Control Systems

Project 2: Deadline-Driven Scheduler

Date	March 31, 2026
Group Members	Nicole Chan → V01009561
	Evy Wind-Granot → V01013467

Project 2: Deadline-Driven Scheduler **1**
1. Introduction 3
2. Design solution 3
2.1. Deadline-Driven Scheduler 3
Figure 1. Chart of how DD Task Scheduler Works 4
2.1.1. DD-Task Data Structure 5
Figure 2. DD-Task Structure 5
2.1.2. DD-Task List 5
Figure 3. DD-Task-List Structure 5
Figure 4. Flow of DD-Task sorting 6
2.1.3. Message Queue and interface 6
Figure 5. Queue Messages Structure 7
2.1.4. Priority Breakdown 7
Figure 6. taskListUpdatePriorites() method 7
Table 1: F-Tasks' Priority Levels 8
2.2. User-Defined Tasks 8
Figure 7. The Task parameters (For tasks 2) 8
2.3. DD-Task Generator 9
Figure 8. Task generator 9
Figure 9. Flow of DD-Task Generator 10
2.4. Monitor Task 10
Figure 10. Monitor_Task() 11
2.5. System Overview 11
Table 2. Queue Architecture 11
2.6. Initial Design Document & Comparison to Final Design 11
2.6.1. The Initial Design 11
Figure 11. Initial design document 12
2.6.2 Differences between Initial and Final Designs 13
3. Discussion 13
4. Limitations and Possible Improvements 14
5. Summary 15
6. Appendix 15
Table 1: DDS Events for Test Bench #1 15
Table 2: Monitor Task Output for Test Bench #2 after One Hyper-Period (1500 ms). 16

1. Introduction

The aim of this project is to design a custom Deadline-Driven Scheduler (DDS) to efficiently manage tasks with hard deadlines in a dynamic manner using FreeRTOS tasks, queues, and timers. This DDS is based on the Earliest Deadline First (EDF) algorithm, which dynamically manages a set of periodic user-defined tasks by controlling their FreeRTOS priorities.

FreeRTOS does not provide support for EDF scheduling, so this DDS is designed with FreeRTOS priority-based preemptive scheduling algorithm. The tasks managed by the DDS are aliased as DD-Tasks, and tasks managed by FreeRTOS will be known as F-Tasks. This DDS uses a little bit of a different priority structure of the DD-Task to the earliest absolute deadline, thus simulating EDF scheduling for FreeRTOS.

The system is comprised of four primary components:

- Deadline-Driven Scheduler (DDS): An F-Task scheduling using EDF and dynamically managing the priorities of user-defined F-Tasks from a maintained list of DD-Tasks.
- User-Defined Tasks: These are periodic F-Tasks that execute user-defined busy-loop workload and signal their completion to the DDS.
- DD-Task Generators: These are software timer callbacks that periodically generate new DD-Task instances by sending messages to the DDS.
- Monitor Task: This is a periodic timer callback that queries the DDS and reports the number of active, completed, and overdue DD-Tasks.

2. Design solution

2.1. Deadline-Driven Scheduler

The Deadline-Driven Scheduler(DDS) is realized by a high-priority FreeRTOS task (DDTaskScheduler) that waits indefinitely on a message queue (xQueue_Msg_To_DDS) until an interface function puts a message into the waiting queue, processes the message, and updates its internal task lists and re-adjusts the priorities of the user tasks before resuming the waiting process.

The DDS uses three singly-linked lists of DD-Tasks:

- Active Task List: The set of DD-Tasks that have been released but not yet completed.
- Completed Task List: The set of DD-Tasks that completed execution before their absolute deadline.
- Overdue Task List: The set of DD-Tasks that completed execution after their absolute deadline.

Each of the three lists has a head node with a zeroed dd_task field to simplify the handling of an empty list.

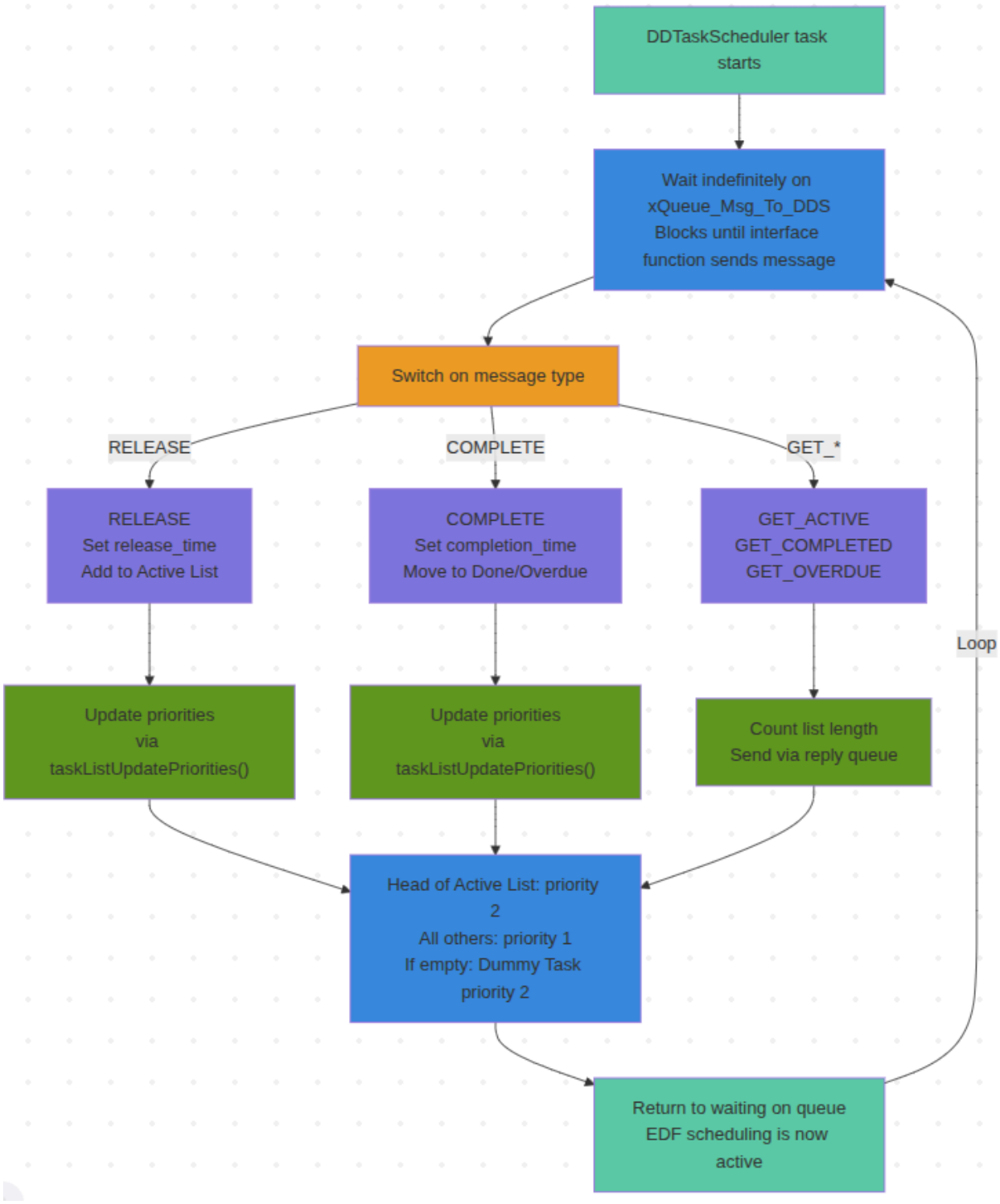

Figure 1. Chart of how DD Task Scheduler Works

2.1.1. DD-Task Data Structure

The structure of dd_task used is the one found in the template of the lab manual, demonstrated below:

```
typedef struct dd_task {
    TaskHandle_t t_handle;
    task_type type;
    uint32_t task_id;
    uint32_t release_time;
    uint32_t absolute_deadline;
    uint32_t completion_time;
} dd_task;
```

Figure 2. DD-Task Structure

In this structure, the release_time and completion_time fields are filled in by the DDS at the moment the release or complete messages, respectively, are processed, ensuring high timing accuracy.

2.1.2. DD-Task List

The data type or structure for all three task lists is merely a singly linked list, as suggested by the lab manual.

```
typedef struct dd_task_list {
    dd_task task;
    struct dd_task_list* next_task;
} dd_task_list;
```

Figure 3. DD-Task-List Structure

The Active Task List is a sorted list with ordering in ascending order following the absolute_deadline. The insertion of a node in the list is done by invoking the insertionSort() method. As expected, the insertion traverses through the list and splices in the node in the correct position. The completed list and overdue list are simply inserted in FIFO order as no ordering is required.

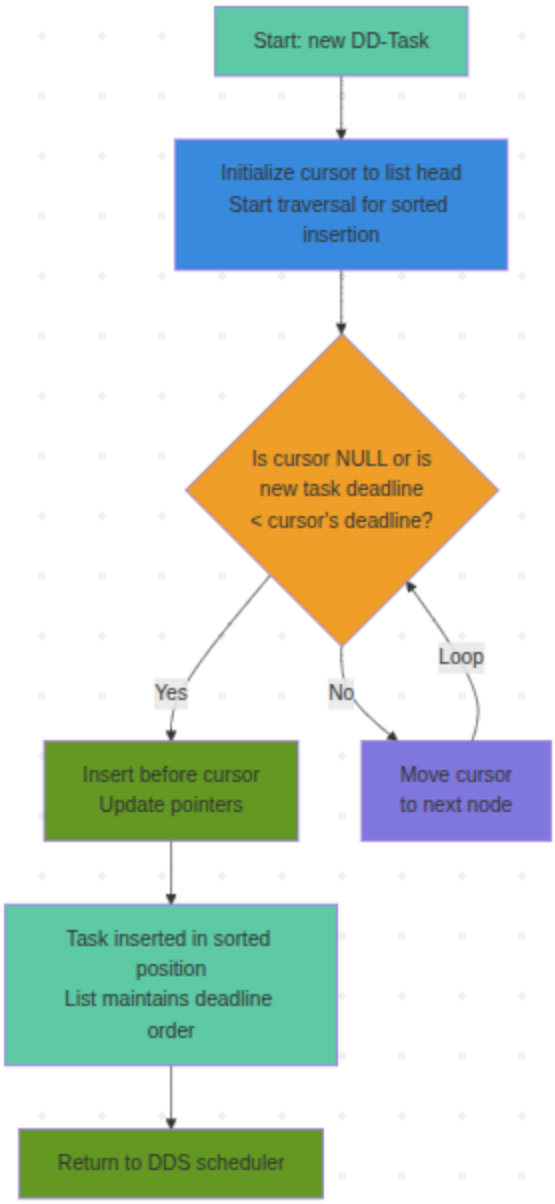

Figure 4. Flow of DD-Task sorting

2.1.3. Message Queue and interface

Communication with DDS happens through a single queue, xQueue_Msg_To_DDS, which uses queueMessage data structure found below. There are 5 messages supported by DDS, which are identified by dds_interface_call.

```
typedef struct queueMessage {
    dds_interface_call interface_call;
    dd_task task;
    uint32_t id;
} queueMessage;
```

Figure 5. Queue Messages Structure

The five interface functions along with their behavior are as follows:

- release_dd_task(): This function sends a new dd_task in a RELEASE message, but without release_time and completion_time, to xQueue_Msg_To_DDS. The DDS receives the task, sets the release_time, and adds the task to the sorted Active List, changing priorities appropriately.
- complete_dd_task(): This function sends a COMPLETE message with only the task_id to the DDS. The DDS gets the COMPLETE message then it sets the completion_time, and removes the task from the Active List, adding it instead to the Completed List or the Overdue List considering if completion_time <= absolute_deadline.
- get_active_dd_task_list_length(): This function sends a GET_ACTIVE message and waits on xQueue_Active_Tasks then it gets the int from the DDS which represents the count of tasks.
- get_complete_dd_task_list_length(): This function sends a GET_COMPLETED message and waits on xQueue_Completed_Tasks to receive an integer from the DDS representing the count of tasks in the Completed List.
- get_overdue_dd_task_list_length(): This function sends a GET_OVERDUE message and waits on xQueue

2.1.4. Priority Breakdown

Once a release or complete event is received, the DDS updates the priorities using the taskListUpdatePriorities() method, which uses the EDF mechanism discussed earlier.

```
void taskListUpdatePriorities(dd_task_list* listHead){
    vTaskPrioritySet(task1Handle, idleTaskPriority);
    vTaskPrioritySet(task2Handle, idleTaskPriority);
    vTaskPrioritySet(task3Handle, idleTaskPriority);

    if (listHead->next_task != NULL) {
        vTaskPrioritySet(dummyTaskHandle, lowPriority);
        vTaskPrioritySet(listHead->next_task->task.t_handle, activeTaskPriority);
    } else {
        vTaskPrioritySet(dummyTaskHandle, activeTaskPriority);
    }
}
```

Figure 6. taskListUpdatePriorites() method

The head of the sorted Active List is always a DD-Task with the earliest deadline. Elevating the priority of this task to activeTaskPriority (2) forces the FreeRTOS scheduler to run this task exclusively over all other user tasks, which are running with idleTaskPriority (1). If the Active List is empty, then the Dummy Task is elevated to keep FreeRTOS with a running task above the idle hook, this protects the CPU timing.

The complete F-Task priority table is shown in the following figure:

Table 1: F-Tasks' Priority Levels

F-Task	Priority Level
DD Scheduler (DDTaskScheduler)	3 → taskSchedulerPriority
Dummy Task (idle placeholder)	2 → activeTaskPriority (when no active DD-Task)
Active User Task (Task 1-3)	2 → activeTaskPriority (elevated by DDS)
Idle User Tasks (Task 1-3)	1 → idleTaskPriority
Monitor Task (timer callback)	5 → monitorTaskPriority (FreeRTOS timer daemon)

2.2. User-Defined Tasks

Three F-Tasks are created and run. Each F-Tasks performs a busy-wait, which simulates a fixed execution time, this is followed by a call to complete_dd_task().

TM_DelayMillis() is a calibrated volatile spin-loop providing millisecond-accurate delays based on the clock frequency of the STM32F4 board. This approach has been taken on in place of using FreeRTOS blocking primitives since vTaskDelay() in a user task would unwantingly get the CPU to other tasks mid-execution, breaking EDF principle. Therefore the aforementioned method is invalid.

The three task parameters are declared as constants, execution, periodicity and period:

```
// Task parameters
#define task1_execution 95
#define task1_periodicity PERIODIC
#define task1_period 250

#define task2_execution 150
#define task2_periodicity PERIODIC
#define task2_period 500

#define task3_execution 250
#define task3_periodicity PERIODIC
#define task3_period 750
```

Figure 7. The Task parameters (For tasks 2)

Dummy Task performs a short spin-loop and runs at activeTaskPriority when no DD-Tasks are active, ensuring FreeRTOS has something to schedule above the built-in idle task.

2.3. DD-Task Generator

The task generation for this DDS system is done using three periodic FreeRTOS timers, each of these times are auto-reload and a period in referference to the one set for that bench.

```
xTimers[0] = xTimerCreate("T1 Timer", pdMS_TO_TICKS(task1_period), pdTRUE, (void *) 1, DDTaskGenerator);
xTimers[1] = xTimerCreate("T2 Timer", pdMS_TO_TICKS(task2_period), pdTRUE, (void *) 2, DDTaskGenerator);
xTimers[2] = xTimerCreate("T3 Timer", pdMS_TO_TICKS(task3_period), pdTRUE, (void *) 3, DDTaskGenerator);
```

Figure 8. Task generator

All of the timers have a single callback to DDTaskGenerator(), which checks the timer ID and in doing so, knows which task to release. It is only called once manually at the start to release all the instances at a time 0.

This approach is a single shared callback and it is equivalent to the dedicating of one generator per task that is also mentioned in the lab manual, but since it reduced the number of F-Tasks this was employed.

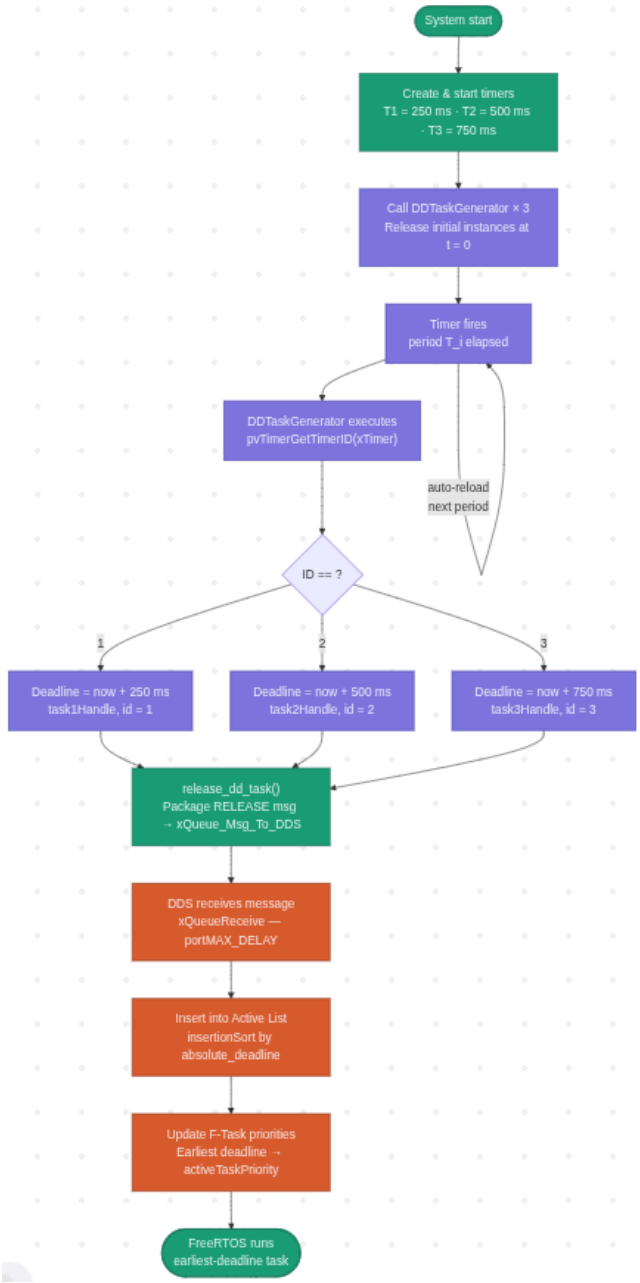

Figure 9. Flow of DD-Task Generator

2.4. Monitor Task

The implementation of the Monitor Task is a periodic FreeRTOS software timer callback, "Monitor_Task," which is triggered every 1000 ms. It calls the interface functions whose purpose is to retrieve the three list length values from the DDS and print them to console:

```c
static void Monitor_Task(TimerHandle_t xTimer) {
    int active = get_active_dd_task_list_length();
    int complete = get_complete_dd_task_list_length();
    int overdue = get_overdue_dd_task_list_length();
    printf("Monitor: Active: %d, Done: %d, Overdue: %d\n", active, complete, overdue);
}
```

Figure 10. Monitor_Task()

Having the monitor task to be a periodic software timer callback, rather than a FreeRTOS task, frees one task stack in RAM, eliminating any priority-related problems with the EDF scheduling. The FreeRTOS task that implements the timer daemon in this configuration (monitorTaskPriority = 5) is higher than the DDS scheduler priority, guaranteeing that monitor print operations are always done without the possibility of influence from user tasks.

2.5. System Overview

The system uses four queues to decouple all inter-component communication

Table 2. Queue Architecture

Queue	Item Type	Purpose
xQueue_Msg_To_DDS	queueMessage	All inbound messages to the DDS (RELEASE, COMPLETE, GET *)
xQueue_Active_Tasks	int	DDS reply to GET_ACTIVE queries
xQueue_Completed_Tasks	int	DDS reply to GET_COMPLETED queries
xQueue_Overdue_Tasks	int	DDS reply to GET_OVERDUE queries

2.6. Initial Design Document & Comparison to Final Design

2.6.1. The Initial Design

As prescribed by the lab manual and teaching team, before a single line of code was written, a one page design document was made to give a direction on how the software components would behave and interact. The document dives into the data structures, queues, tasks and interface function involved in the DDS.

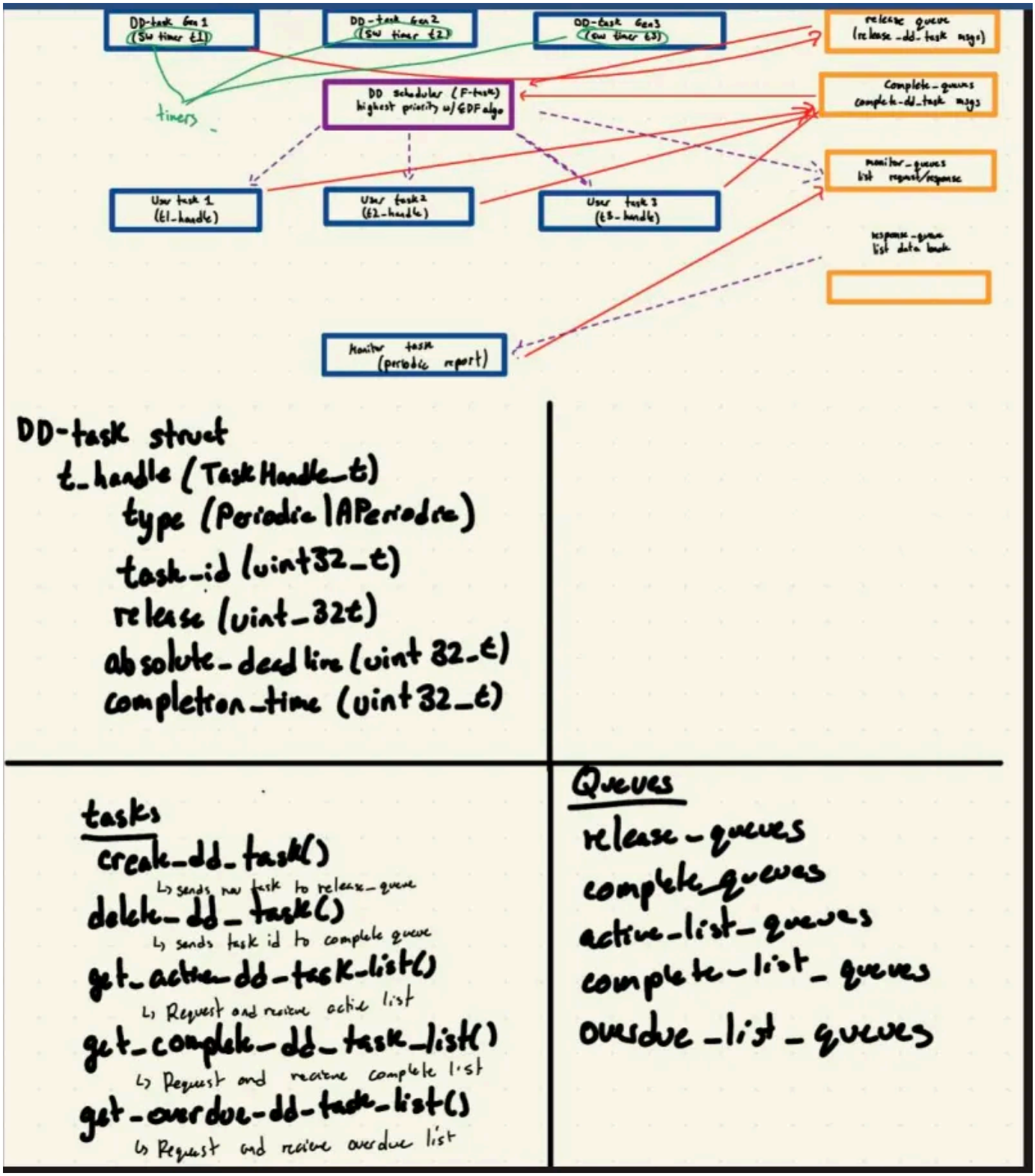

Figure 11. Initial design document

The initial design document computed the major elements of the final system including:

- The DD-Task struct: This structure and choice was directly taken from the lab manuals specification, so no changes were required for the final version.
- The major F-Tasks: This includes the three DD-Task generators, the DD scheduler as the highest priority F-Task with the EDF scheduling algorithm, three user-tasks, and the monitoring task.
- Additionally all the DDS interface functions, create_dd_task(), delete_dd_task(), get_active_task_list(), get_complete_dd_task_list(), get_overdue_dd_task_list(). All of the functionality of these tasks is found in the figure above.
- The five queues, the release and complete queues, and the three list response queues for the active, complete and overdue tasks.

2.6.2 Differences between Initial and Final Designs

There were certainly many similarities between the initial and final versions, yet also some differences here are the main ones:

- Queue consolidation and messages: In the initial design, the release and complete queues were two separated queues, inbound channels to the DDS. In the final design these were merged into a single xQueue_Msg_To_DDS queue which holds the tagged queueMessage struct with a dds_interface_call enum field. The little modification of lowering the number of queues means that the DDS does not need to poll as many times and it ensures that a FIFO ordering is maintained for all message types.
- Adding the Dummy Task: The initial implementation of the system did not include the much necessary dummy task, for idle-period scheduling. In hindsight this oversight was quite strange as clearly when the Active Task List is empty all three of the user tasks sit at idel at priority 1. Without this dummy taks, the FreeRTOS idle hook would run, which was interfering with the timing. This built-in idle hook task has unpredictable execution, and introduces jitter into measurements, has some priority inversion problems, and has some serious chache/ context switching overhead. All of these reasons make it unreasonable not to include some dummy task. So this simple and harmless dummy dask was added to take place when no other task was there. This was a clean method to entertain the CPU in the times where it would be idle otherwise.

So, overall the initial design provided a good blueprint, but some other functions and functionalities were necessary to fully make the system behave as it should, which was to be expected.

3. Discussion

This project implements a scheduling mechanism implemented by the DDS to dynamically change the priorities of periodically or aperiodically generated tasks so that the correct task is executed. This scheduling mechanism ensures that the task with the earliest deadline is assigned the higher priority, and the rest of the tasks are given a lower priority. Using EDF scheduling, the scheduler only executes a single task which is the task with the earliest deadline.

In Test Bench #1, the CPU utilization is around 82%, so it's comfortably below the 100% limit for EDF scheduling. As there is quite a lot of room to run, all tasks met their deadlines over the 1500 ms hyper-period which was expected. The observed timing closely matched the theoretical EDF schedule,

with only small deviations of a few milliseconds. These differences are most likely due to queue communication overhead and the time it takes to update task priorities inside the scheduler.

Test Bench #2 pushes the system slightly over the limit, with a utilization of about 101%. In this test bench, Task 3 occasionally misses its deadline, and this aligns with the EDF scheduling mechanism we implemented. Once the total utilization exceeds 100%, no scheduling algorithm can guarantee all deadlines will be met. The Monitor Task successfully detected and reported these overdue tasks, which confirms that the system's tracking of completed and missed deadlines is working correctly.

Test Bench #3 represents a boundary case, with utilization right at 100%. Most tasks were still schedulable, but small timing variations occasionally caused tasks to be flagged as slightly overdue. This would be due to interference from the Monitor Task, which runs at a higher priority and can interrupt the scheduler. Even though these delays are minor, they highlight how sensitive EDF can be when operating at full utilization.

One design choice that worked particularly well was implementing the Monitor Task as a software timer callback instead of a regular FreeRTOS task. This avoids allocating an extra task stack and ensures that monitoring always runs at a high priority. As a result, monitoring output is reliable and doesn't risk being delayed or blocked by other tasks.

Another improvement over the initial design was consolidating communication into a single message queue. Using one queue with a tagged message structure made the scheduler logic simpler and easier to extend, compared to managing multiple queues.

4. Limitations and Possible Improvements

There are some areas where the current design could be improved. First, the system currently reuses the same task handles for each period. If a task has not finished before its next release, this can lead to conflicts or incorrect scheduling. To make this more robust, the system could be modified to dynamically create and delete tasks each period. However, this could possibly add additional memory overhead.

Another limitation is overdue task detection, as it only occurs when the next release event happens. When multiple tasks are released at the same time, delays in message handling can cause jitter. In systems with tighter timing constraints, this could result in overdue tasks not being detected until an entire period later.

It was also observed that the results from Test Bench 3 were somewhat inconsistent. The utilization is right at 100%, which causes the system to run with no timing margin, so even tiny variations in execution time or scheduling overhead can push a task over its deadline. This resulted in some runs completing without any missed deadlines, and others flagging tasks as overdue. This unpredictability at the boundary is difficult to fully remove.

Finally, the lists that track completed and overdue tasks aren't bounded, so they just keep growing over time. Systems that run for a long time, could eventually use up all available memory. Solutions for this would be to either limit the size of these lists or clear them periodically.

5. Summary

This project successfully designs a custom DDS to dynamically manage tasks which have hard execution deadlines. The DDS uses EDF scheduling to maximize processor utilization. Since FreeRTOS is not configurable to perform deadline-driven scheduling, the DDS is built on top of theFreeRTOS task scheduler. The system dynamically adjusts task priorities based on deadlines and uses a centralized message queue to coordinate scheduling decisions.

The design consists of four main parts: the DDS itself, three user tasks that simulate workload and signal completion, a task generator that periodically releases new jobs, and a monitor that reports system status. Using the three test benches during testing showed that the system behaves as expected: when utilization is below or equal to 100%, deadlines are generally met, and when it exceeds 100%, missed deadlines will occur.

6. Appendix

Table 1: DDS Events for Test Bench #1

Event #	Event	Expected Time (ms)	Event #
1	Task 1 released	0	1
2	Task 2 released	0	2
3	Task 3 released	0	3
4	Task 1 complete	95	4
5	Task 2 complete	245	5
6	Task 3 complete	495	6
7	Task 1 released	500	7
8	Task 2 released	500	8
9	Task 1 complete	595	9

10	Task 2 complete	745	10
11	Task 3 released	750	11
12	Task 3 complete	1000	12
13	Task 1 released	1000	13
14	Task 2 released	1000	14
15	Task 1 complete	1095	15
16	Task 2 complete	1245	16
17	Task 1 released	1500	17
18	Task 2 released	1500	18

Table 2: Monitor Task Output for Test Bench #2 after One Hyper-Period (1500 ms)

Metric	Measured	Expected
Number of active DD-Tasks	0	0
Number of completed DD-Tasks	10	9–10
Number of overdue DD-Tasks	1	1

main.c

```c
/* Standard includes. */
#include <stdint.h>
#include <stdio.h>
/* Kernel includes. */
#include "stm32f4xx.h"
#include "../FreeRTOS_Source/include/FreeRTOS.h"
#include "../FreeRTOS_Source/include/queue.h"
#include "../FreeRTOS_Source/include/semphr.h"
#include "../FreeRTOS_Source/include/task.h"
#include "../FreeRTOS_Source/include/timers.h"
#include "../FreeRTOS_Source/include/projdefs.h"

// Task parameters
#define task1_execution 95
#define task1_periodicity PERIODIC
#define task1_period 250

#define task2_execution 150
#define task2_periodicity PERIODIC
#define task2_period 500

#define task3_execution 250
#define task3_periodicity PERIODIC
#define task3_period 750

#ifndef pdTICKS_TO_MS
    #define pdTICKS_TO_MS( xTimeInTicks )    ( ( TickType_t ) ( ( ( uint64_t ) ( xTimeInTicks ) * ( uint64_t ) 1000U ) / ( uint64_t ) configTICK_RATE_HZ ) )
#endif

#define Queue_Length 5
#define Queue_Timeout 20
#define wait 100000

typedef enum task_type {PERIODIC, APERIODIC} task_type;
typedef enum dds_interface_call {RELEASE, COMPLETE, GET_ACTIVE, GET_COMPLETED, GET_OVERDUE} dds_interface_call;

typedef struct dd_task {
    TaskHandle_t t_handle;
    task_type type;
    uint32_t task_id;
    uint32_t release_time;
    uint32_t absolute_deadline;
    uint32_t completion_time;
} dd_task;

typedef struct dd_task_list {
    dd_task task;
    struct dd_task_list* next_task;
} dd_task_list;

typedef struct queueMessage {
    dds_interface_call interface_call;
    dd_task task;
    uint32_t id;
} queueMessage;
```

```c

void release_dd_task(TaskHandle_t t_handle, task_type type, uint32_t task_id, uint32_t absolute_deadline);
void complete_dd_task(uint32_t task_id);
int get_active_dd_task_list_length(void);
int get_complete_dd_task_list_length(void);
int get_overdue_dd_task_list_length(void);

static void prvSetupHardware(void);
static void Task_1_Handler(void *pvParameters);
static void Task_2_Handler(void *pvParameters);
static void Task_3_Handler(void *pvParameters);
static void Dummy_Task(void *pvParameters);

static void Monitor_Task(TimerHandle_t xTimer);
static void DDTaskGenerator(TimerHandle_t xTimer);
static void DDTaskScheduler(void *pvParameters);

void TM_Delay_Init(void);
void TM_DelayMillis(uint32_t millis);

// Priorities
#define monitorTaskPriority 5
#define taskSchedulerPriority 3
#define activeTaskPriority 2
#define idleTaskPriority 1
#define lowPriority 1

xQueueHandle xQueue_Msg_To_DDS = 0;
xQueueHandle xQueue_Active_Tasks = 0;
xQueueHandle xQueue_Completed_Tasks = 0;
xQueueHandle xQueue_Overdue_Tasks = 0;

TaskHandle_t task1Handle;
TaskHandle_t task2Handle;
TaskHandle_t task3Handle;
TaskHandle_t dummyTaskHandle;
TaskHandle_t taskScheduler;

TimerHandle_t xTimers[3];
TimerHandle_t monitorTimer;

uint32_t timeBegin;
uint32_t delayMultiplier;
```

```c

int main(void)
{
    TM_Delay_Init();
    prvSetupHardware();

    xQueue_Msg_To_DDS = xQueueCreate(Queue_Length, sizeof(queueMessage));
    xQueue_Completed_Tasks = xQueueCreate(Queue_Length, sizeof(int));
    xQueue_Active_Tasks = xQueueCreate(Queue_Length, sizeof(int));
    xQueue_Overdue_Tasks = xQueueCreate(Queue_Length, sizeof(int));

    xTaskCreate(DDTaskScheduler, "Task Scheduler", configMINIMAL_STACK_SIZE * 2, NULL, taskSchedulerPriority, &taskScheduler);
    xTaskCreate(Task_1_Handler, "Task1", configMINIMAL_STACK_SIZE, NULL, idleTaskPriority, &task1Handle);
    xTaskCreate(Task_2_Handler, "Task2", configMINIMAL_STACK_SIZE, NULL, idleTaskPriority, &task2Handle);
    xTaskCreate(Task_3_Handler, "Task3", configMINIMAL_STACK_SIZE, NULL, idleTaskPriority, &task3Handle);
    xTaskCreate(Dummy_Task, "Dummy Task", configMINIMAL_STACK_SIZE, NULL, activeTaskPriority, &dummyTaskHandle);

    xTimers[0] = xTimerCreate("T1 Timer", pdMS_TO_TICKS(task1_period), pdTRUE, (void *) 1, DDTaskGenerator);
    xTimers[1] = xTimerCreate("T2 Timer", pdMS_TO_TICKS(task2_period), pdTRUE, (void *) 2, DDTaskGenerator);
    xTimers[2] = xTimerCreate("T3 Timer", pdMS_TO_TICKS(task3_period), pdTRUE, (void *) 3, DDTaskGenerator);

    xTimerStart(xTimers[0], 0);
    xTimerStart(xTimers[1], 0);
    xTimerStart(xTimers[2], 0);

    monitorTimer = xTimerCreate("Monitor Timer", pdMS_TO_TICKS(1000), pdTRUE, (void *) 1, Monitor_Task);
    xTimerStart(monitorTimer, 0);

    timeBegin = pdTICKS_TO_MS(xTaskGetTickCount());

    DDTaskGenerator(xTimers[0]);
    DDTaskGenerator(xTimers[1]);
    DDTaskGenerator(xTimers[2]);

    vTaskStartScheduler();
}

void TM_Delay_Init(void) {
    RCC_ClocksTypeDef RCC_Clocks;
    RCC_GetClocksFreq(&RCC_Clocks);
    delayMultiplier = RCC_Clocks.HCLK_Frequency / 7000;
}

void TM_DelayMillis(uint32_t millis) {
    volatile uint32_t millis_copy = millis * delayMultiplier - 10;
    while (millis_copy--);
}
```

```c
void insertionSort(dd_task_list* listHead, dd_task newItem){
    dd_task_list* curListNode = listHead;
    while (curListNode->next_task != NULL && curListNode->next_task->task.absolute_deadline <= newItem.absolute_deadline) {
        curListNode = curListNode->next_task;
    }
    dd_task_list* newList = pvPortMalloc(sizeof(dd_task_list)); // Changed to FreeRTOS malloc
    newList->task = newItem;
    newList->next_task = curListNode->next_task;
    curListNode->next_task = newList;
}

void taskListUpdatePriorities(dd_task_list* listHead){
    vTaskPrioritySet(task1Handle, idleTaskPriority);
    vTaskPrioritySet(task2Handle, idleTaskPriority);
    vTaskPrioritySet(task3Handle, idleTaskPriority);

    if (listHead->next_task != NULL) {
        vTaskPrioritySet(dummyTaskHandle, lowPriority);
        vTaskPrioritySet(listHead->next_task->task.t_handle, activeTaskPriority);
    } else {
        vTaskPrioritySet(dummyTaskHandle, activeTaskPriority);
    }
}

```

```c
static void DDTaskScheduler(void *pvParameters){
    queueMessage receivedMessage;
    dd_task_list activeTaskListHead = {{0}, NULL};
    dd_task_list completeTaskListHead = {{0}, NULL};
    dd_task_list overdueTaskListHead = {{0}, NULL};
    int activeTaskListLength = 0;
    int completeTaskListLength = 0;
    int overdueTaskListLength = 0;
    uint32_t currentTime;

    while(1){
        if (xQueueReceive(xQueue_Msg_To_DDS, &receivedMessage, portMAX_DELAY)){
            switch(receivedMessage.interface_call) {
                case RELEASE:
                    currentTime = pdTICKS_TO_MS(xTaskGetTickCount()) - timeBegin;
                    receivedMessage.task.release_time = currentTime;
                    insertionSort(&activeTaskListHead, receivedMessage.task);
                    activeTaskListLength++;
                    taskListUpdatePriorities(&activeTaskListHead);
                    break;
                case COMPLETE:
                    currentTime = pdTICKS_TO_MS(xTaskGetTickCount()) - timeBegin;
                    if (activeTaskListHead.next_task != NULL) {
                        dd_task_list* prevList = &activeTaskListHead;
                        dd_task_list* curList = activeTaskListHead.next_task;
                        while (curList != NULL && curList->task.task_id != receivedMessage.id){
                            prevList = curList;
                            curList = curList->next_task;
                        }
                        if(curList != NULL) {
                            prevList->next_task = curList->next_task;
                            curList->task.completion_time = currentTime;
                            activeTaskListLength--;
                            if(curList->task.absolute_deadline >= curList->task.completion_time){
                                curList->next_task = completeTaskListHead.next_task;
                                completeTaskListHead.next_task = curList;
                                completeTaskListLength++;
                            } else {
                                curList->next_task = overdueTaskListHead.next_task;
                                overdueTaskListHead.next_task = curList;
                                overdueTaskListLength++;
                            }
                            taskListUpdatePriorities(&activeTaskListHead);
                            printf("ID: %d, Rel: %d, Done: %d, Dead: %d\n", (int)curList->task.task_id, (int)curList->task.release_time, (int)
                            curList->task.completion_time, (int)curList->task.absolute_deadline);
                        }
                    }
                    break;
                case GET_ACTIVE:
                    xQueueSend(xQueue_Active_Tasks, &activeTaskListLength, Queue_Timeout);
                    break;
                case GET_COMPLETED:
                    xQueueSend(xQueue_Completed_Tasks, &completeTaskListLength, Queue_Timeout);
                    break;
                case GET_OVERDUE:
                    xQueueSend(xQueue_Overdue_Tasks, &overdueTaskListLength, Queue_Timeout);
                    break;
```

```c
            }
        }
    }
}

```

```c
static void DDTaskGenerator(TimerHandle_t xTimer){
    uint32_t timerId = (uint32_t) pvTimerGetTimerID(xTimer);
    uint32_t deadline = pdTICKS_TO_MS(xTaskGetTickCount()) - timeBegin;

    if (timerId == 1){
        release_dd_task(task1Handle, task1_periodicity, 1, task1_period + deadline);
    } else if (timerId == 2){
        release_dd_task(task2Handle, task2_periodicity, 2, task2_period + deadline);
    } else if (timerId == 3){
        release_dd_task(task3Handle, task3_periodicity, 3, task3_period + deadline);
    }
}

void release_dd_task(TaskHandle_t t_handle, task_type type, uint32_t task_id, uint32_t absolute_deadline){
    dd_task new_task = {t_handle, type, task_id, 0, absolute_deadline, 0};
    queueMessage new_message = {RELEASE, new_task, 0};
    xQueueSend(xQueue_Msg_To_DDS, &new_message, Queue_Timeout);
}

void complete_dd_task(uint32_t task_id){
    queueMessage new_message = {COMPLETE, {NULL}, task_id};
    xQueueSend(xQueue_Msg_To_DDS, &new_message, Queue_Timeout);
}

int get_active_dd_task_list_length(void){
    queueMessage msg = {GET_ACTIVE, {NULL}, 0};
    int active = 0;
    xQueueSend(xQueue_Msg_To_DDS, &msg, Queue_Timeout);
    xQueueReceive(xQueue_Active_Tasks, &active, Queue_Timeout);
    return active;
}

int get_complete_dd_task_list_length(void){
    queueMessage msg = {GET_COMPLETED, {NULL}, 0};
    int complete = 0;
    xQueueSend(xQueue_Msg_To_DDS, &msg, Queue_Timeout);
    xQueueReceive(xQueue_Completed_Tasks, &complete, Queue_Timeout);
    return complete;
}

int get_overdue_dd_task_list_length(void){
    queueMessage msg = {GET_OVERDUE, {NULL}, 0};
    int overdue = 0;
    xQueueSend(xQueue_Msg_To_DDS, &msg, Queue_Timeout);
    xQueueReceive(xQueue_Overdue_Tasks, &overdue, Queue_Timeout);
    return overdue;
}

static void Monitor_Task(TimerHandle_t xTimer) {
    int active = get_active_dd_task_list_length();
    int complete = get_complete_dd_task_list_length();
    int overdue = get_overdue_dd_task_list_length();
    printf("Monitor: Active: %d, Done: %d, Overdue: %d\n", active, complete, overdue);
}
```

```c
static void Task_1_Handler(void *pvParameters) {
    while(1) {
        TM_DelayMillis(task1_execution);
        complete_dd_task(1);
    }
}

static void Task_2_Handler(void *pvParameters) {
    while(1) {
        TM_DelayMillis(task2_execution);
        complete_dd_task(2);
    }
}

static void Task_3_Handler(void *pvParameters) {
    while(1) {
        TM_DelayMillis(task3_execution);
        complete_dd_task(3);
    }
}

static void Dummy_Task(void *pvParameters) {
    while(1) {
        for(volatile int i = 0; i < wait; ++i);
    }
}

static void prvSetupHardware(void) {
    NVIC_SetPriorityGrouping(0);
}

// Error hooks
void vApplicationMallocFailedHook(void) { for(;;); }
void vApplicationStackOverflowHook(xTaskHandle pxTask, signed char *pcTaskName) { for(;;); }
void vApplicationIdleHook(void) { }
```